

Chapter 01 파이썬 기초 01



강의개요

- 강의목적

- ✓ 위성영상의 기초 이론과 이미지 처리 기술에 대한 이해를 제공합니다. 수강생들은 프로그래밍 실습을 통해 위성영상 처리하고, 데이터 분석 및 시각화 기술을 익힐 수 있습니다.
- ✓ 위성영상 분야에서의 최신 연구 동향을 탐구하고, 문제 해결 능력을 향상시키는데 도움이 됩니다.
- ✓ 본 수업은 Python 프로그래밍 코딩 학습을 통해 기초적인 위성영상처리 및 활용에 관한 실습을 수행합니다.

- 위성원격탐사 기초 이론 : 실습 병행 = 1 : 3

- Python 프로그래밍 실습

- 개인노트북 지참 필요 (노트북이 없는 학생은 수업 전에 대여 가능)
 - 강의 교재는 실습 과목이라 구매 필요 없음
 - ! Python 프로그래밍 기초는 **점프 투 파이썬 위키독스** 자료 최대한 활용
- <https://wikidocs.net/book/1>

강의개요

• 시험

- ✓ 중간고사는 손코딩 시험
- ✓ 기말고사는 프로그래밍 프로젝트 시험 (위성자료처리, 코딩 및 분석)
 - 조별 위성영상처리 및 활용 관련 프로젝트
 - 산불 탐지, 구름탐지
 - 조는 2인 1조 씩: (추후 예정)

• 기타강의 공지사항

- 실습 과목이라 출석 점수가 높게 반영 (20%)
 - 학칙에 따라 출석인정
 - 출석인정은 전문의 진단서 반영여부 결정 (! 단순 진료 및 병결은 인정 X)
 - 코로나로 인한 병결 인정 (관련 서류 제출 필요)
- 목요일은 8 교시만 수업 진행 예정
- **수업태도 10% 반영**
 - 수업방해, 수업 중 영상 시청, 개인 메신저 및 SNS, 수업 외 다른 행위 시 감점
 - 1회당 5점 감점

강의계획서 (2024년 2학기)

교과목명	위성영상처리및실습	분반	1	담당교수명	염승민
교과목 코드	0000128493	학점	3	연락처	063-270-3350
교과목 구분	전공선택	요일/시간	화 6-A, 화 6-B, 화 7-A, 화 7-B, 목 8-A, 목 8-B, 목 9-A, 목 9-B	E-mail	jmyeom@jbnu.ac.kr
학과/학년	지구환경과학 2	강의실	전주:자연과학대학4동관 114호 전주:자연과학대학4동관 21	연구실	자연대 4호관 207호
				상당가능시간	수업시간에 미리 상당 가능일정을 잡아주세요

1. 강의 기본정보

수업목표	위성영상의 기초 이론과 이미지 처리 기술에 대한 이해를 제공합니다. 수강생들은 프로그램 및 실습을 통해 위성영상처리, 데이터 분석 및 시각화 기술을 익힐 수 있습니다. 또한, 위성영상 분야에서의 최신 연구 동향을 탐구하고, 문제 해결 능력을 향상시키는데 도움이 됩니다.								
직전강의평가 및 CQI반영사항	특이사항 없음								
6대 핵심역량과의 관계									
구분	소통역량	창의역량	인성역량	실무역량	도전역량	문화역량	합계	대표역량	
비율(%)	30	20	0	50	0	0	100		
교과목간의 연계성									
주교재	원격탐사 원리와 방법								
저자	이규성		출판사	씨아이알			출판년도	2021	
참고자료	정크 루 마이센 (박종용, https://wikidocs.net/book/1) 원격탐사와 디지털 영상처리 (John R. Jensen) 원경원격탐사 (John R. Jensen)								
교재언어	한국어		강의언어	한국어		필요 기자재	위성영상처리 실습을 위한 개인 휴대용 노트북, 필요 (노트북이 없는 학생에게 수업시간		
권장 선수과목				권장 후수과목	DA				
수업방식 (복수가능)	강의	발표/토론	PBL	플립러닝	LMS활용	실험실습	기타		
	√					√			
수업운영방향									
평가계획 (100%)	중간	기말	출석	과제물	안전교육	발표/토론	수업태도	기타	
	40%	30%	20%	0%	0%	0%	10%	0%	
평가참고사항									
평가방법	상대평가 II	상대평가 비율	A(%)	A+B(%)		C이하(%)		총비율	
			40	80		20		100%	
		절대평가 기준							
참고 사항	* 장애학생 교수학습지원 사항								
	- 강의	√ 강의 파일, 자료 등 제공			√ 좌석배치(지정좌석) 조정				
	기타 : _____								
	- 과제	과제 제출기한 연장			√ 대안적 과제 제시				
	- 평가	시험시간 연장			√ 평가방법 조정(대독, 구두응답, 도우미 대필, 답안작성 등)				
	√ 별도의 시험 장소 제공								
	기타 : _____								
	그 외(필요시 자유로이 추가 기술) : _____								
	※ 위 지원사항 등을 포함한 강의, 과제, 시험 등 학습과정에서 장애로 인하여 추가 지원이 필요한 경우 개강전 담당강사 및 장애학습 지원센터를 통해 문의 바랍니다.								
	주별 강의내용								
주별	수업목표		수업내용		수업방식	자료, 과제 및 기타 참고사항		수업방식별시간	
								온라인	오프라인
1주	제1장 강의개요 및 목적 설명		강의개요 및 프로그래밍 실습 목적 설명		Python 프로그래밍 코딩 실습	위성 자료를 이용하여 코딩 실습			

주별 강의내용						
주별	수업목표	수업내용	수업방식	자료, 과제 및 기타 참고사항	수업방식별시간	
					온라인	오프라인
2주	제2장 파이썬 기초 및 실습 I	Colab 및 파이썬 설치, 기초 및 자료형	Python 프로그래밍 코딩 실습	위성 자료를 이용하여 코딩 실습		
3주	제3장 파이썬 기초 및 실습 II	프로그래밍 제어문 I	Python 프로그래밍 코딩 실습	위성 자료를 이용하여 코딩 실습		
4주	제4장 파이썬 기초 및 실습 III	프로그래밍 제어문 II	Python 프로그래밍 코딩 실습	위성 자료를 이용하여 코딩 실습		
5주	제5장 파이썬 기초 및 실습 IIII	프로그래밍 입출력	Python 프로그래밍 코딩 실습	위성 자료를 이용하여 코딩 실습		
6주	제6장 파이썬 기초 및 실습 IV	파이썬 클래스 I	Python 프로그래밍 코딩 실습	위성 자료를 이용하여 코딩 실습		
7주	제7장 파이썬 기초 및 실습 V	파이썬 클래스 II	Python 프로그래밍 코딩 실습	위성 자료를 이용하여 코딩 실습		
8주	중간고사	중간고사	-	-		
9주	제8장 위성 영상처리 기초 실습 I	위성영상 입출력 실습	Python 프로그래밍 코딩 실습	위성 자료를 이용하여 코딩 실습		
10주	제9장 위성 영상처리 기초 실습 II	위성영상 구름탐지 실습	Python 프로그래밍 코딩 실습	위성 자료를 이용하여 코딩 실습		
11주	제10장 위성 영상처리 기초 실습 IIII	위성영상 지수산출	Python 프로그래밍 코딩 실습	위성 자료를 이용하여 코딩 실습		
12주	제11장 위성 영상처리 응용실습 I	위성영상 지수산출 응용	Python 프로그래밍 코딩 실습	위성 자료를 이용하여 코딩 실습		
13주	제12장 위성 영상처리 응용실습 II	위성영상 변화탐지 I	Python 프로그래밍 코딩 실습	위성 자료를 이용하여 코딩 실습		
14주	제13장 위성 영상처리 응용실습 IIII	위성영상 변화탐지 II	Python 프로그래밍 코딩 실습	위성 자료를 이용하여 코딩 실습		
15주	기말고사	기말고사	-	-		

강의개요

- 강의를 통해 얻게 되는 결과물

- ✓ Python 프로그래밍 기초

- ✓ 위성영상처리 기초

- 자료 입출력
- 자료처리 및 계산

- ✓ 정지궤도위성 영상처리 응용

- 구름탐지
- 지수산출
- 위성영상 이미지 표출

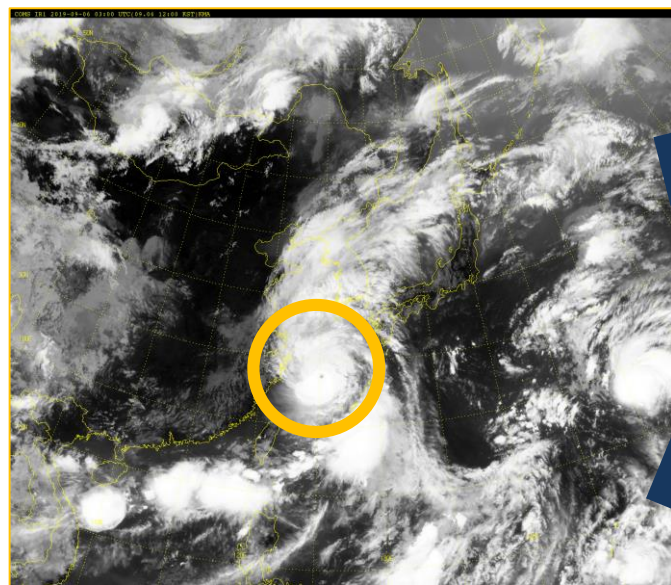
- ✓ 고해상도위성 영상처리 응용

- 지수산출
- 산불탐지
- 위성영상 이미지 표출

강의개요

- COMS/MI 위성 자료를 활용한 태풍 "링링" 탐지

✓ COMS/MI IR1 (11 μ m) 밝기 온도 < 경계값 $\rightarrow$ 구름



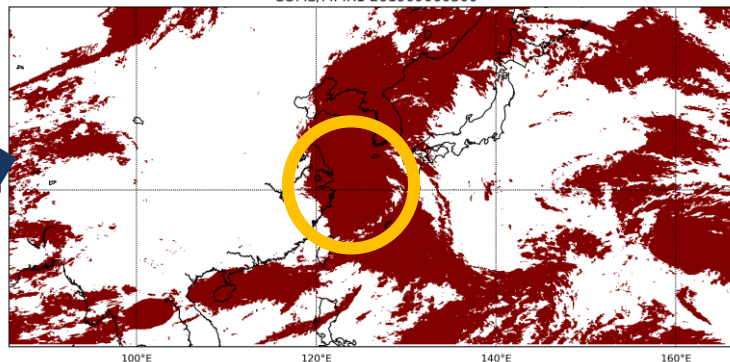
COMS/MI IR1(11 μ m) 영상
<2019-09-06 12:00>

270

220

태풍(구름) 탐지 예시 영상
<2019-09-06 12:00 KST>

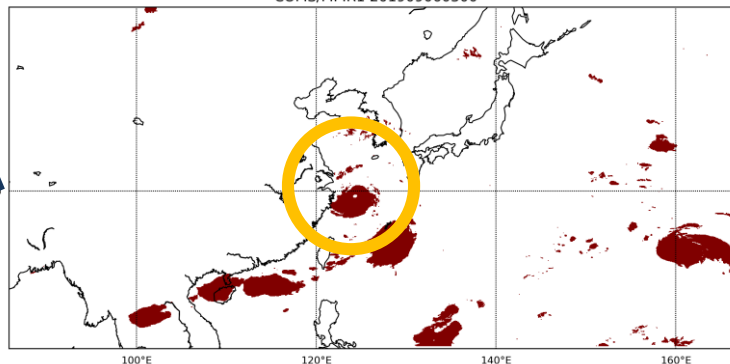
COMS/MI IR1 201909060300



■ 구름

□ 청천

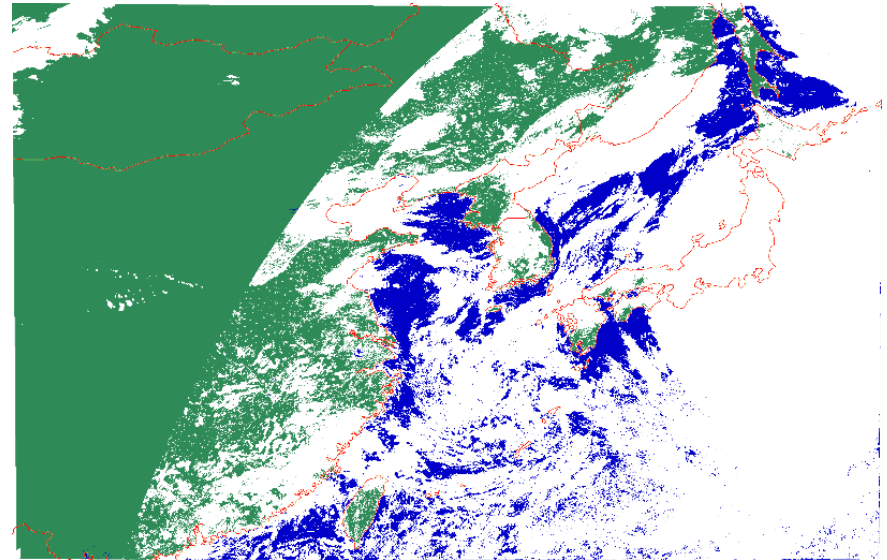
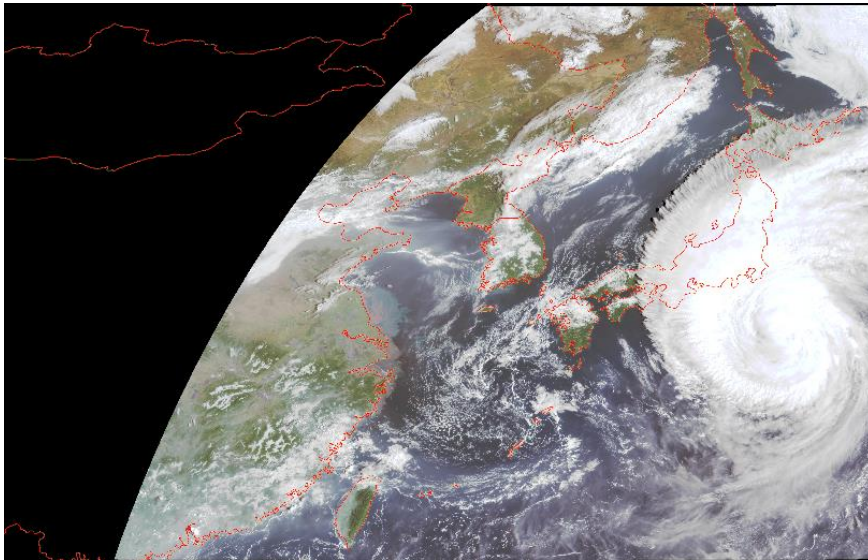
COMS/MI IR1 201909060300



강의개요

- 구름탐지 실습

- GK-2A 구름탐지 실습
 - GK-2A 자료 입력
 - 구름탐지 수행
 - GK-2A 구름탐지 자료 출력



구름

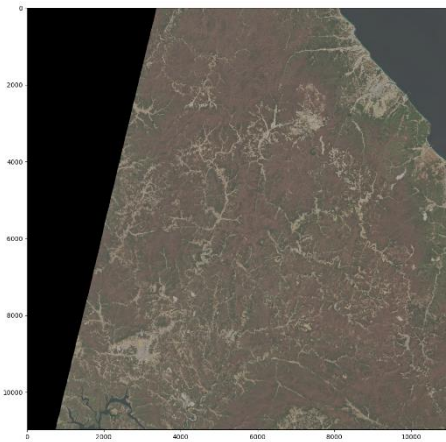
육지

해양

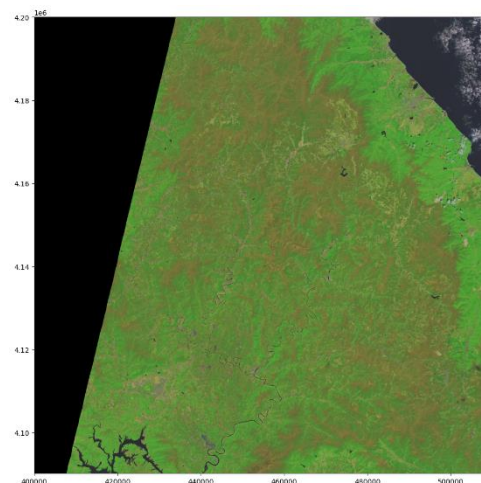
강의개요

- 산불지역 변화 탐지 실습

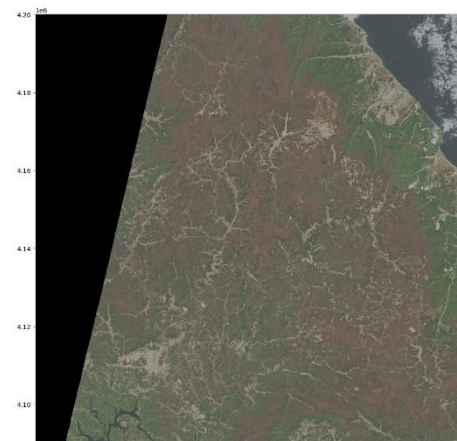
- 2023년 4월 강릉 산불피해 지역 탐지 및 면적 분석 기술 개발



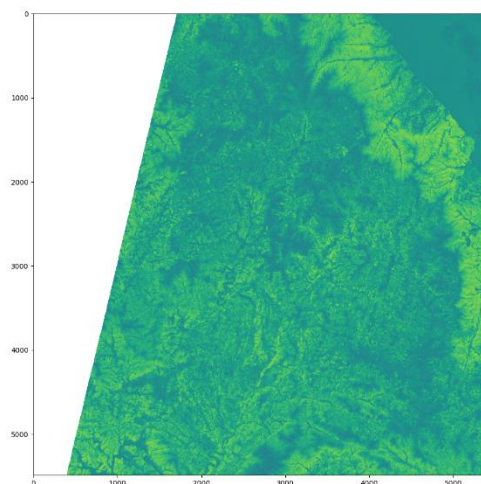
산불 발생 전



False image



산불 발생 후

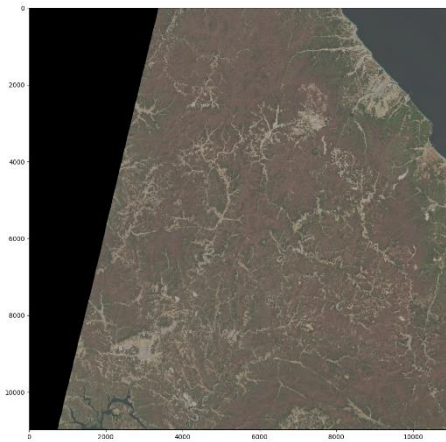


Index image

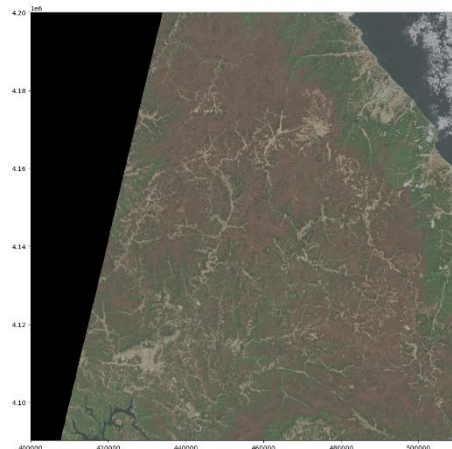
강의개요

- 산불지역 변화 탐지 실습

- 2023년 4월 강릉 산불피해 지역 탐지 및 면적 분석 기술 개발



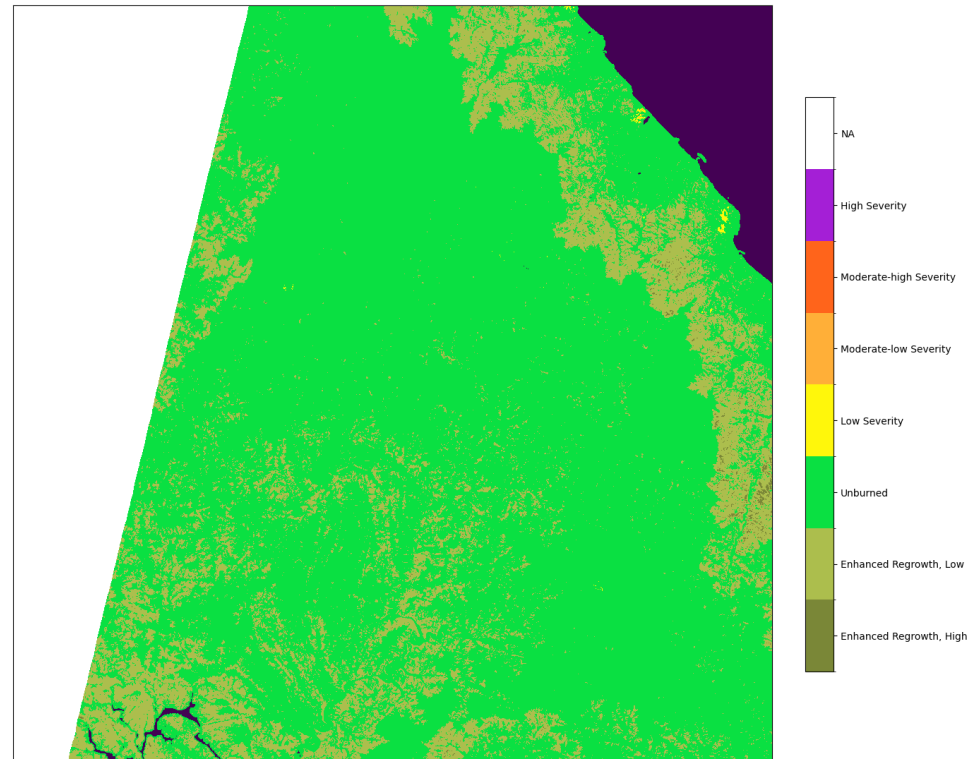
산불 발생 전



산불 발생 후

산불 피해지도 산출

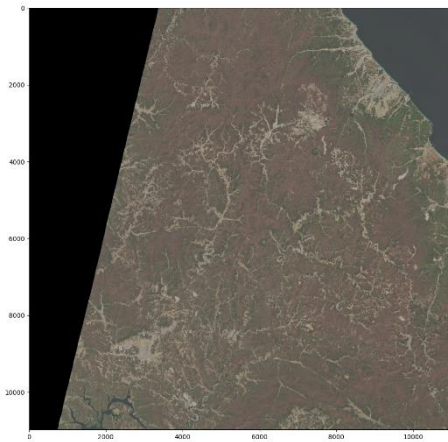
Burn Severity Map - Greece (19 April 2023)



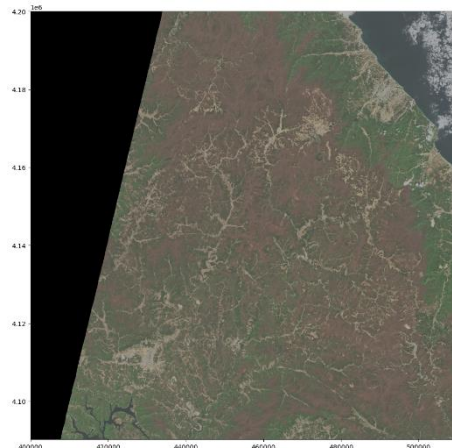
강의개요

- 산불지역 변화 탐지 실습

- 2023년 4월 강릉 산불피해 지역 탐지 및 면적 분석 기술 개발

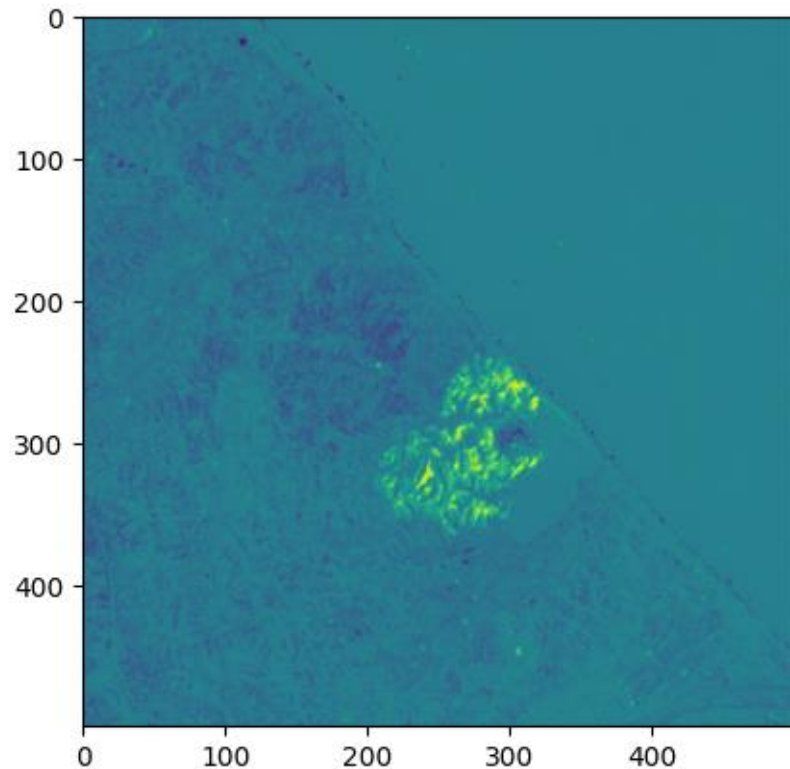


산불 발생 전



산불 발생 후

산불 피해지도 산출



광고 # 1

인공지능 환경원격탐사 연구실 학부연구생 및 대학원생 모집 (9월 30일까지)

• 연구분야

- ✓ 위성자료와 딥러닝을 이용한 재난재해 탐지 및 예측
- ✓ 신재생에너지(태양광 및 풍력) 잠재량 산출 및 예측
- ✓ 기후변화 및 지구환경 감시, 분석, 예측: (농업작황, 식생관측, 산불, 가뭄 등)
- ✓ 다중위성을 이용한 융합지표반사도 산출 및 디지털 트윈 개발
- ✓ 위성자료와 딥러닝을 이용한 지구환경 미래예측 기술 개발

• 모집 과정

- ✓ 석사/박사 또는 석박사 통합과정 대학원생
- ✓ 대학원 진학을 원하는 2, 3 그리고 4학년 학부연구생 모집
- ✓ 전공은 무관 (상당가능)
- ✓ 프로그래밍 코딩 경험 (혹은 수업) 있는 학생 우대

• 지원 (혜택) 내용

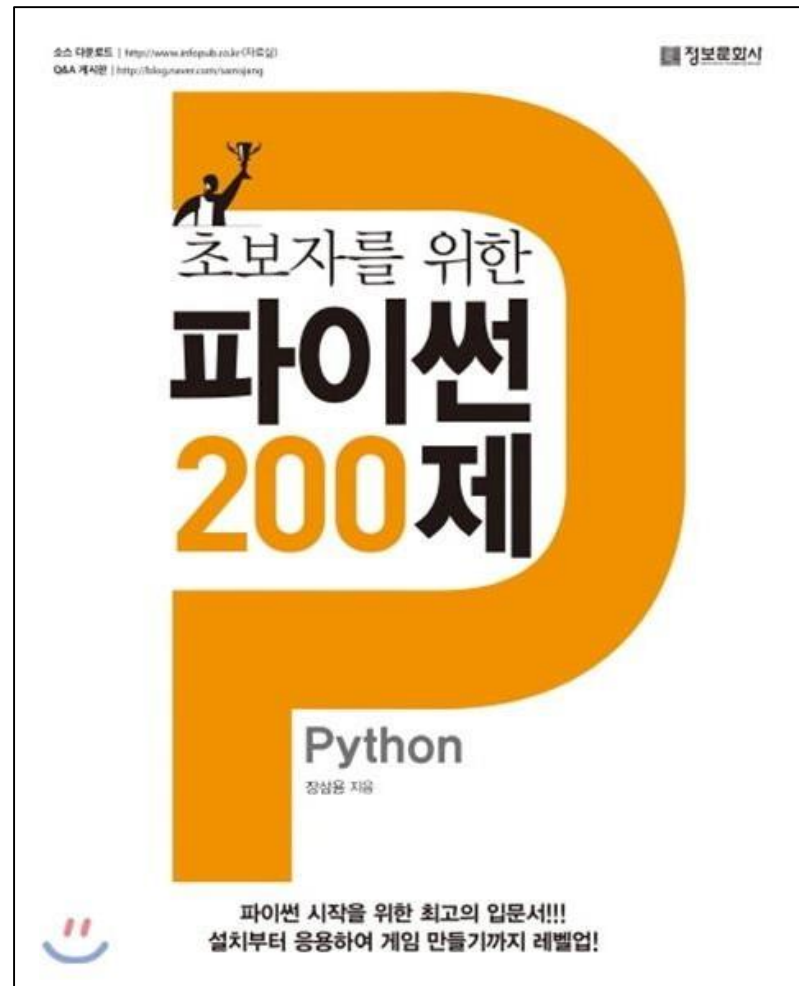
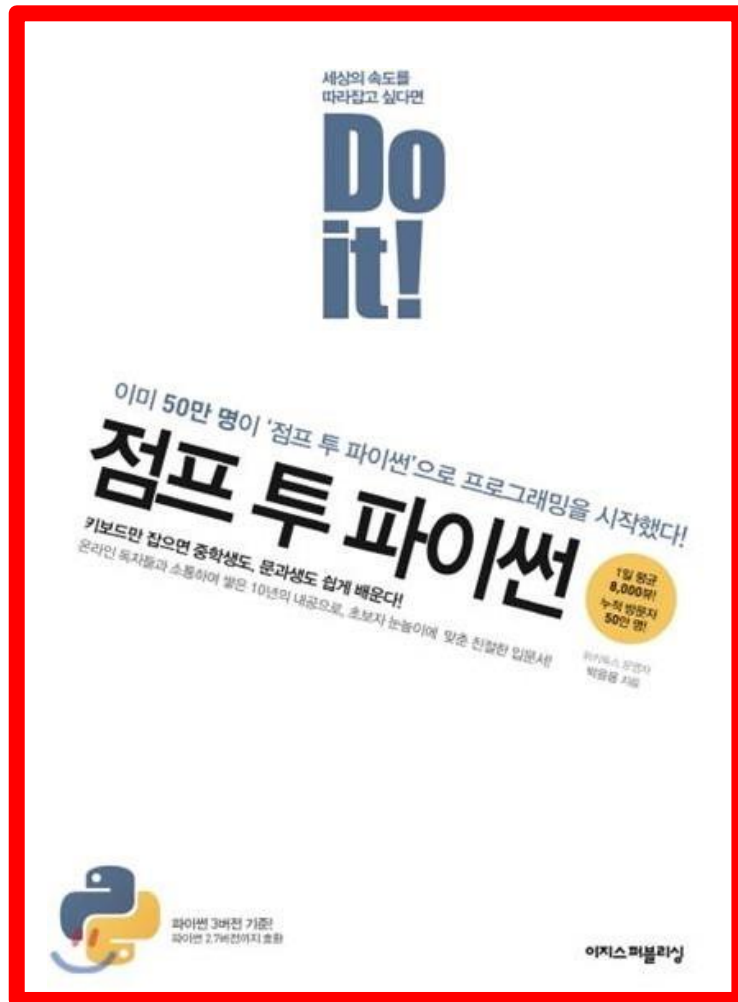
- ✓ 석,박사과정은 학비 전액 무료 및 생활비 지급 (지도교수의 연구예비 산에서 지급)
- ✓ 학부연구생 연구장학금 지급 (지도교수의 연구예비 산에서 지급)
- ✓ 연구 관련 PC 및 연구실 공간 제공 (자연대 4호관 208호, 214호)
- ✓ 연구성과에 따른 국내외 학술대회 참석/해외연구기관(NASA, ESA 등) 방문 연구 지원

• 지원 및 문의

- ✓ 염종민 교수 이메일 (jmyeom@jbnu.ac.kr; yeom.jongmin@gmail.com)로 제출
- ✓ 자기소개서 제출 필요 (자유 양식)

- 파이썬 설치 ? jupyter notebook (딥러닝을 위해서는 고려 필요)
 - Colab을 활용할 예정
- 파이썬 사용 목적 : 정형, 비정형 빅데이터(텍스트, 이미지/영상, GIS)
 - ➔ 통계분석, 웹크롤링, 리눅스, 머신러닝, 딥러닝
 - 문법이 쉬워 빠르게 배울 수 있다
 - 무료이지만 강력하다
 - 간결하다
 - 프로그래밍을 즐기게 해 준다 ?
 - “Life is too short, You need python”
 - ➔ 최근에는 딥러닝 코딩을 위해서는 Python이 필수적으로 사용됨

파이썬 입문서



파이썬 설치



python



검색결과 약 467,000,000개 (0.40초)

Welcome to Python.org

<https://www.python.org> ▾ 이 페이지 번역하기

The official home of the **Python** Programming Language.



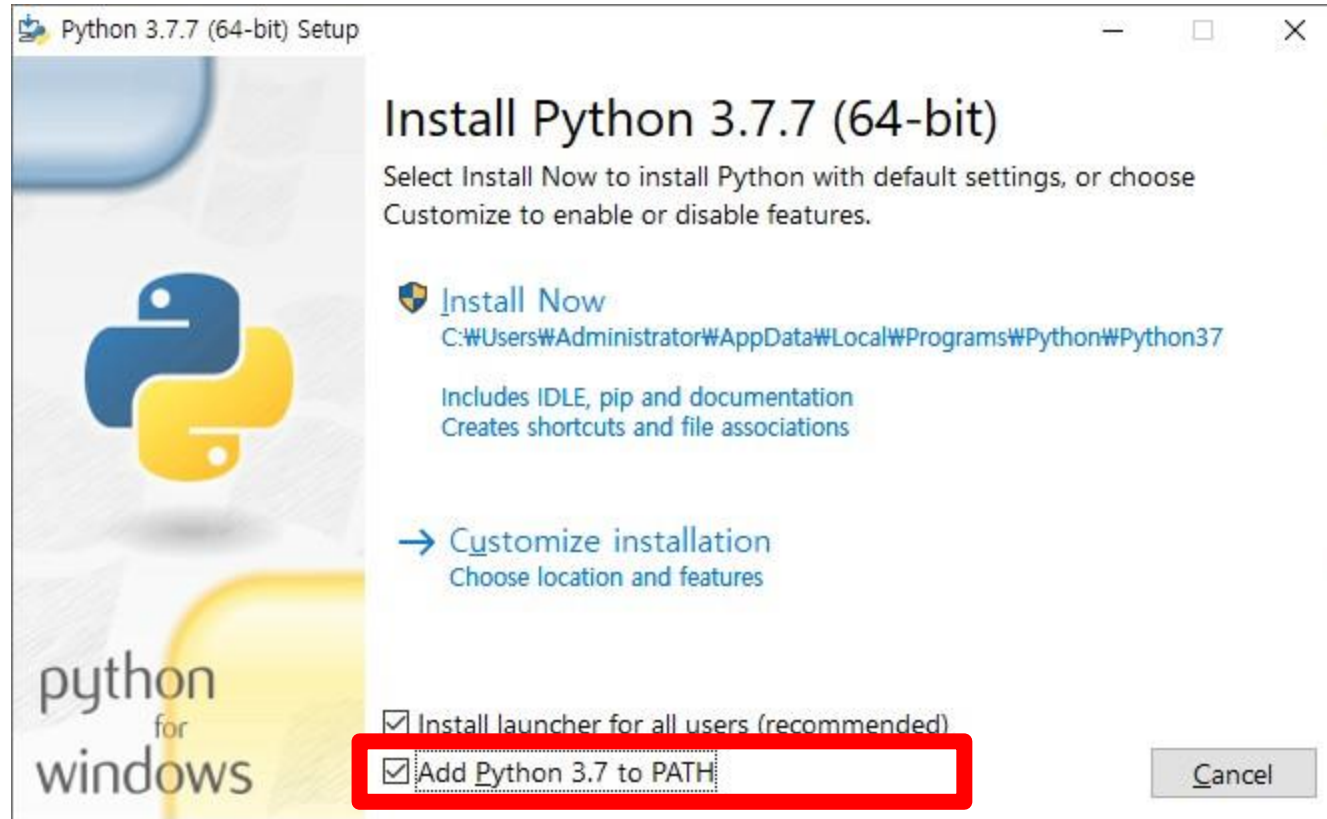
Stable Releases

- [Python 3.7.7 - March 10, 2020](#)

Note that Python 3.7.7 cannot be used on Windows XP or earlier.

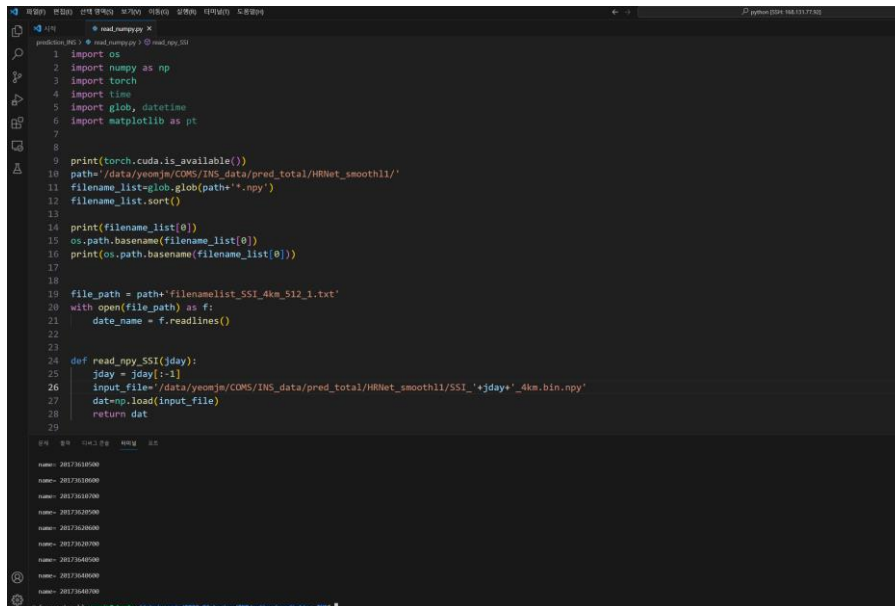
- Download [Windows help file](#)
- Download [Windows x86-64 embeddable zip file](#)
- Download [Windows x86-64 executable installer](#)
- Download [Windows x86-64 web-based installer](#)
- Download [Windows x86 embeddable zip file](#)
- Download [Windows x86 executable installer](#)
- Download [Windows x86 web-based installer](#)

파이썬 설치



파이썬 코딩 환경

1. 비주얼 스튜디오 코드
 2. PyCharm
 3. Colab을 이용한 실습
- ✓ Jupyter Notebook 혹은 Jupyter Lab 활용

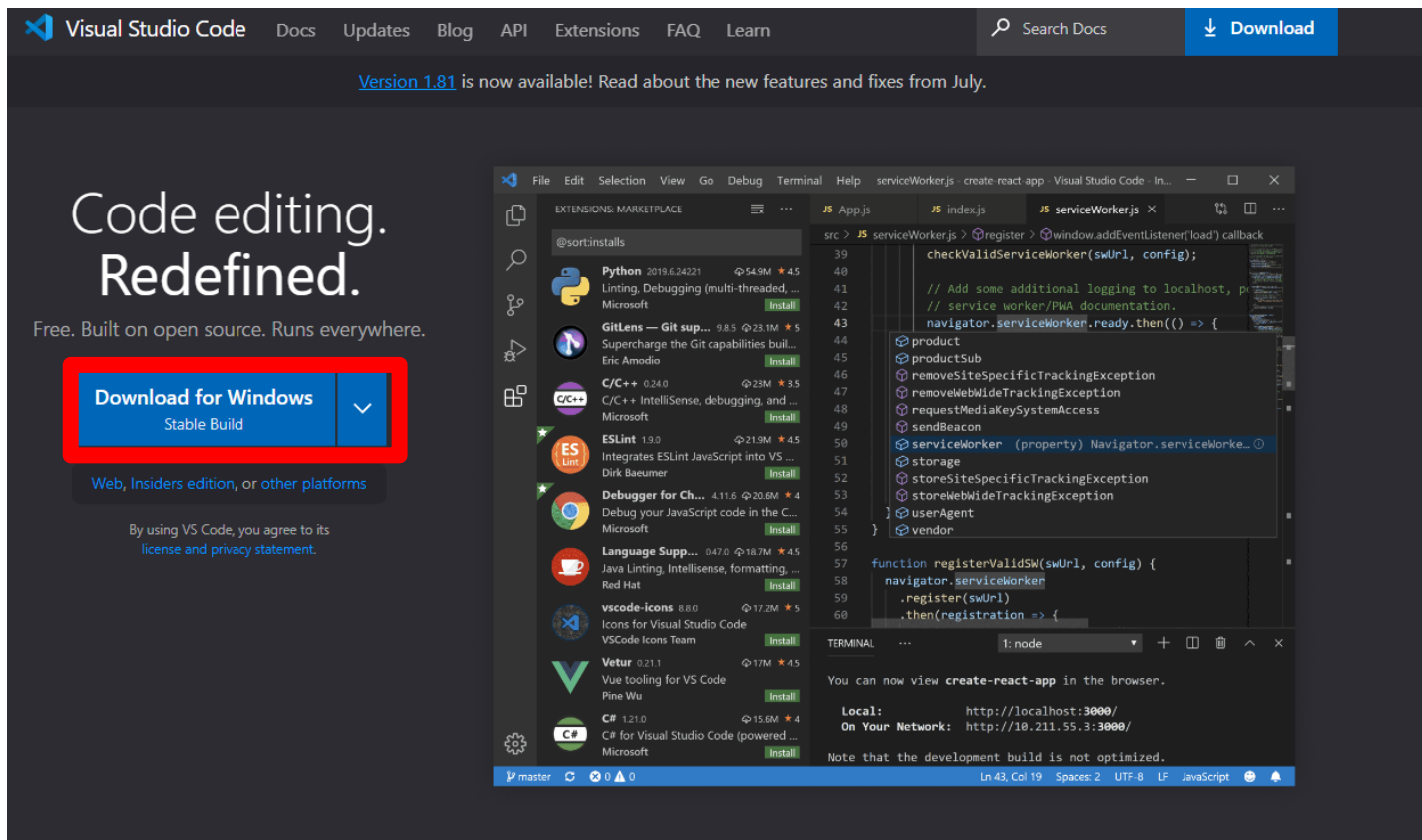


```
1 import os
2 import numpy as np
3 import torch
4 import time
5 import glob, datetime
6 import matplotlib as plt
7
8
9 print(torch.cuda.is_available())
10 path = "/data/yeonjin/COMS/INS_data/pred_total/HRNet_smooth11/"
11 filename_list = glob.glob(path + '*.npy')
12 filename_list.sort()
13
14 print(filename_list[0])
15 os.path.basename(filename_list[0])
16 print(os.path.basename(filename_list[0]))
17
18
19 file_path = path + filename_list[0]
20 with open(file_path) as f:
21     date_name = f.readlines()
22
23
24 def read_npy_SSI(jday):
25     jday = jday[-1]
26     input_file = "/data/yeonjin/COMS/INS_data/pred_total/HRNet_smooth11/SSI_{}_days_4km.bin.npy".format(jday)
27     data = np.load(input_file)
28     return data
29
```



파이썬 코딩 환경

- 비주얼 스튜디오 코드(Visual studio code)는
 - ✓ 비주얼 스튜디오 코드는 공식 다운로드 사이트(code.visualstudio.com)에서 다운로드 가능



IntelliSense



Run and Debug



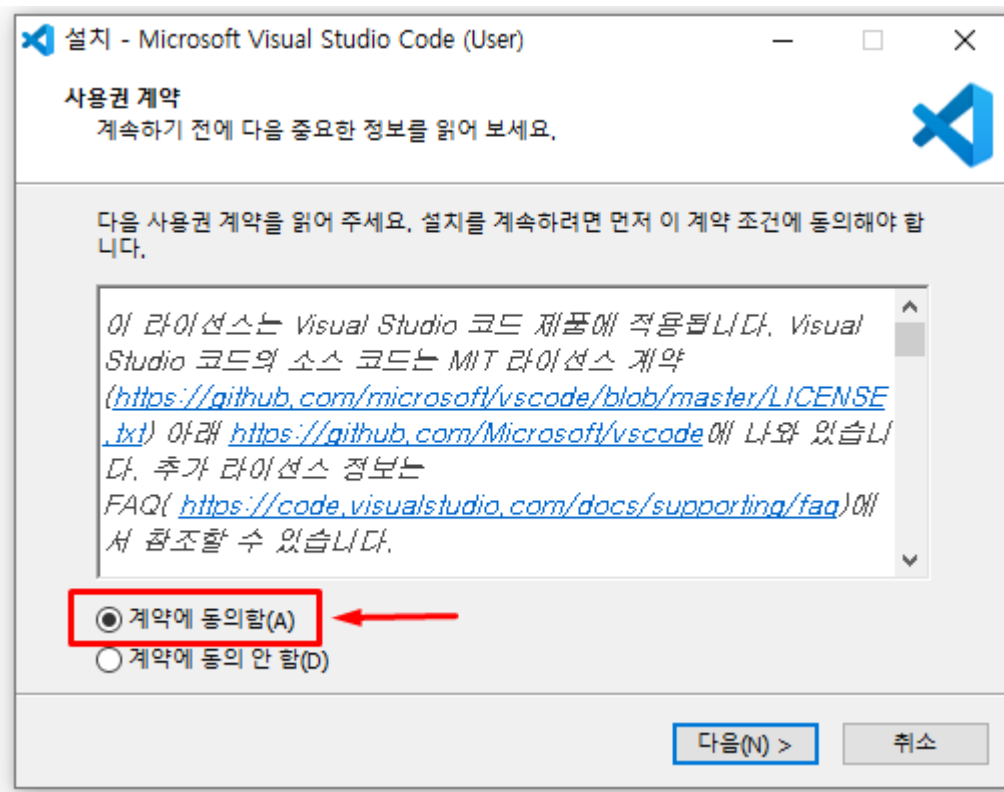
Built-in Git



Extensions

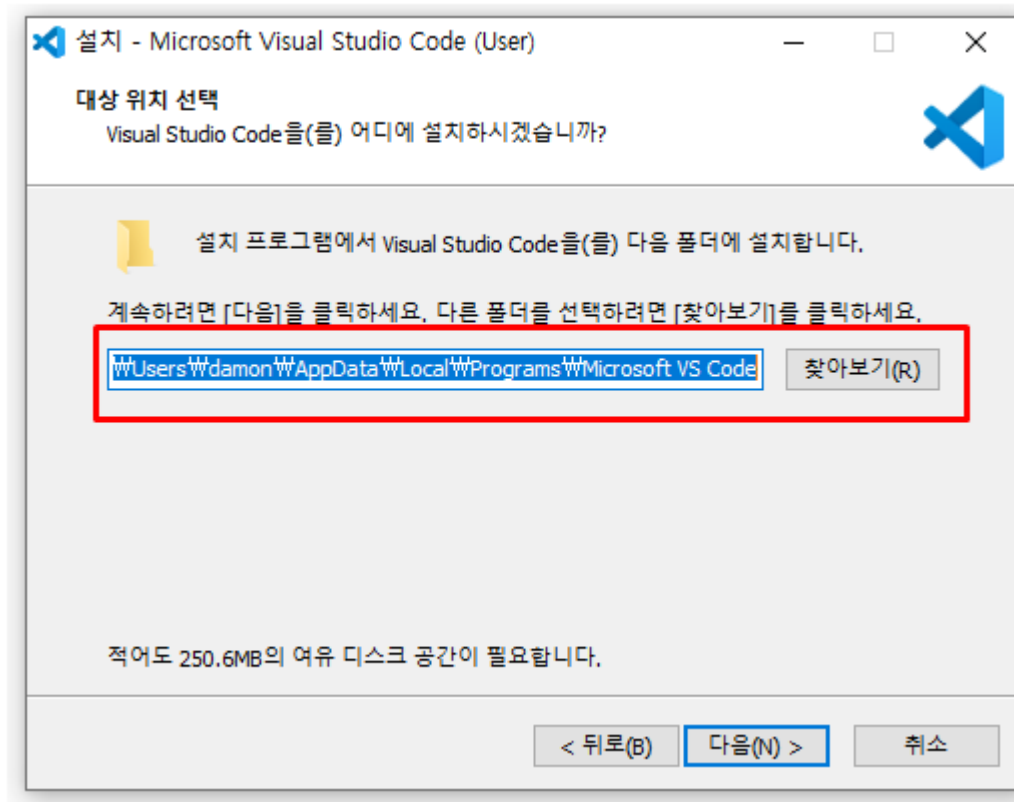
파이썬 코딩 환경

- 사용권 계약 동의



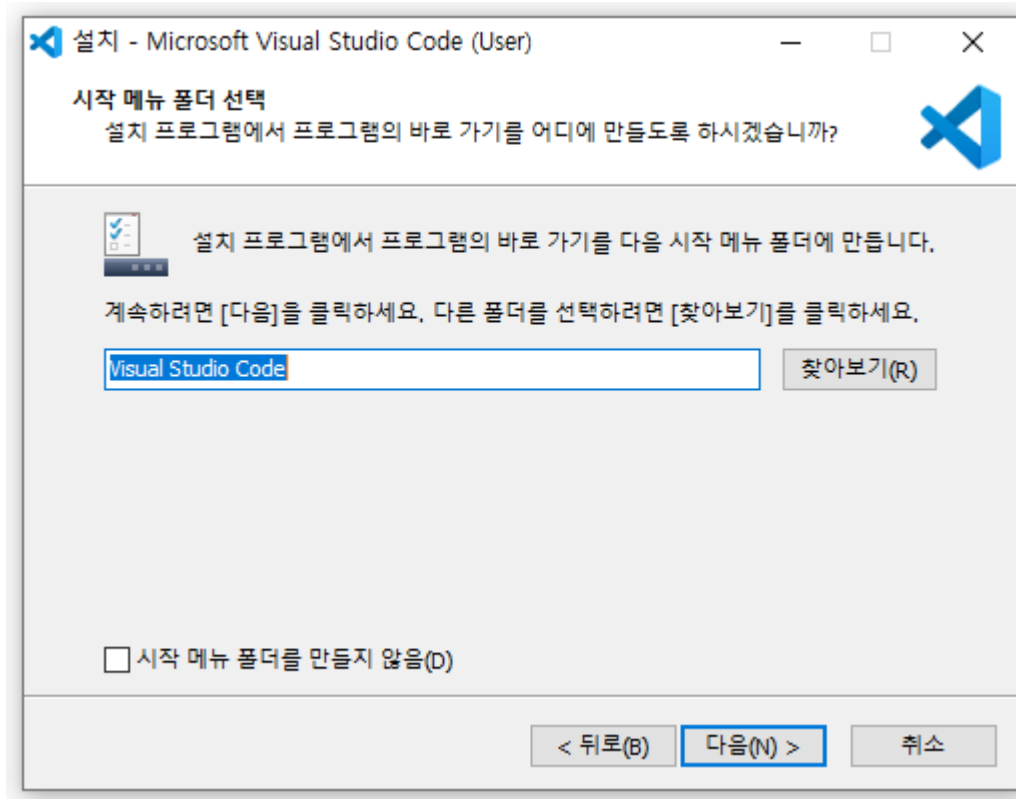
파이썬 코딩 환경

- 설치 경로 확인



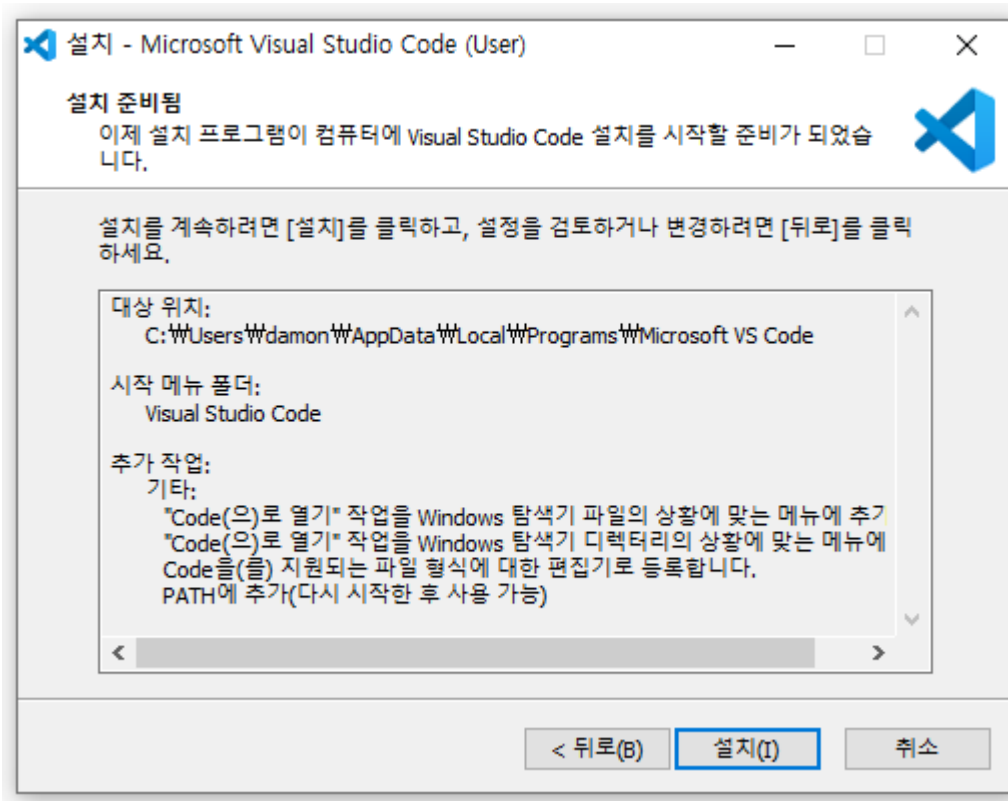
파이썬 코딩 환경

- 시작 메뉴 폴더 선택



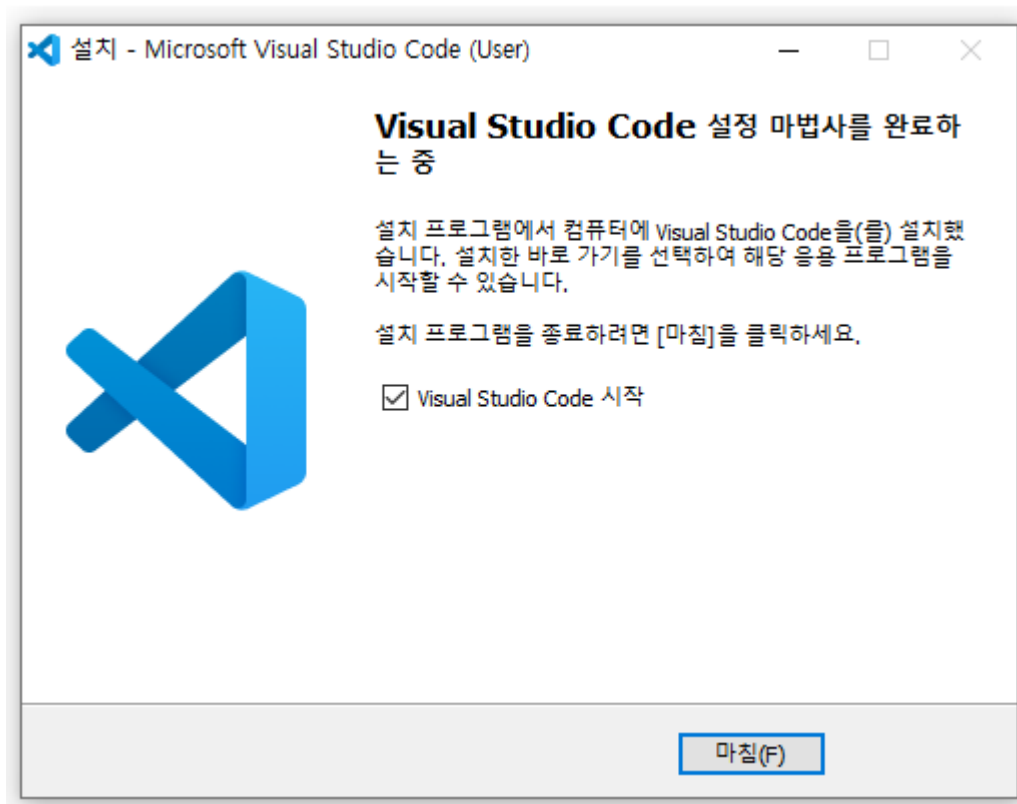
파이썬 코딩 환경

- 설치 시작



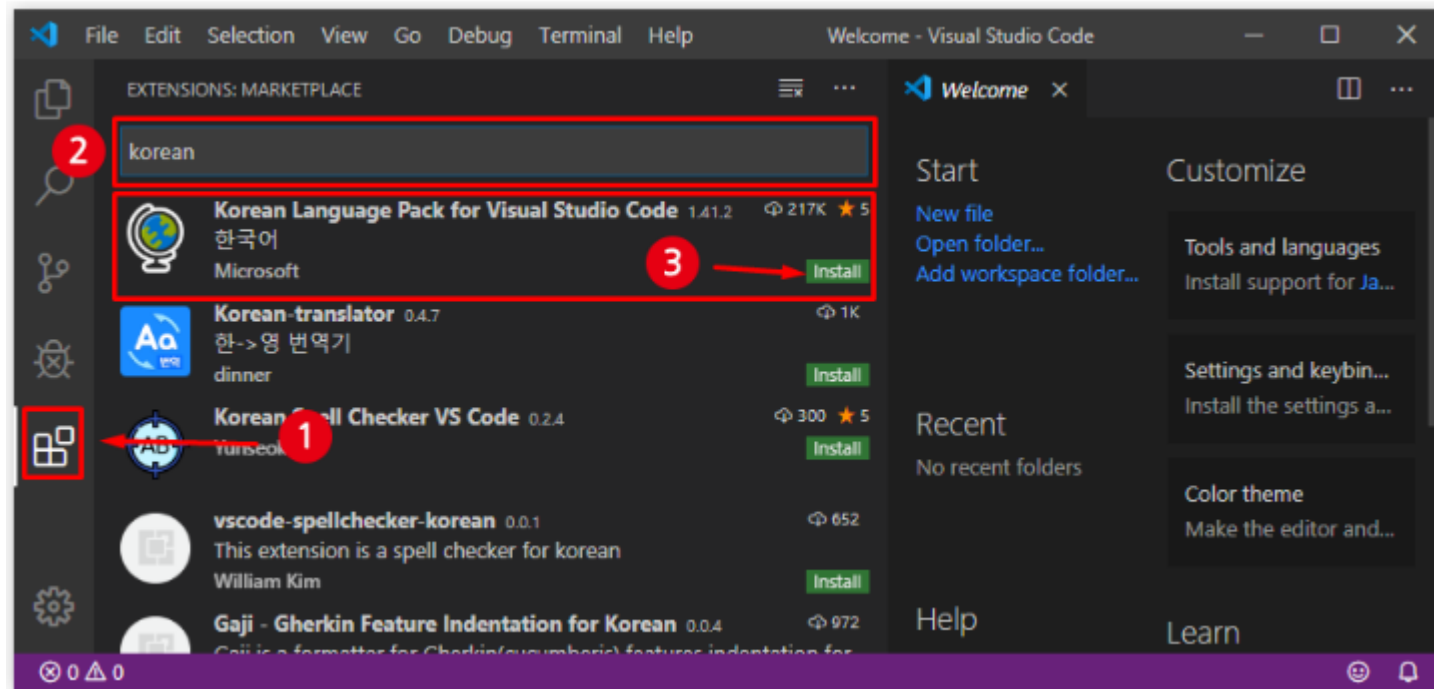
파이썬 코딩 환경

- 설치 완료



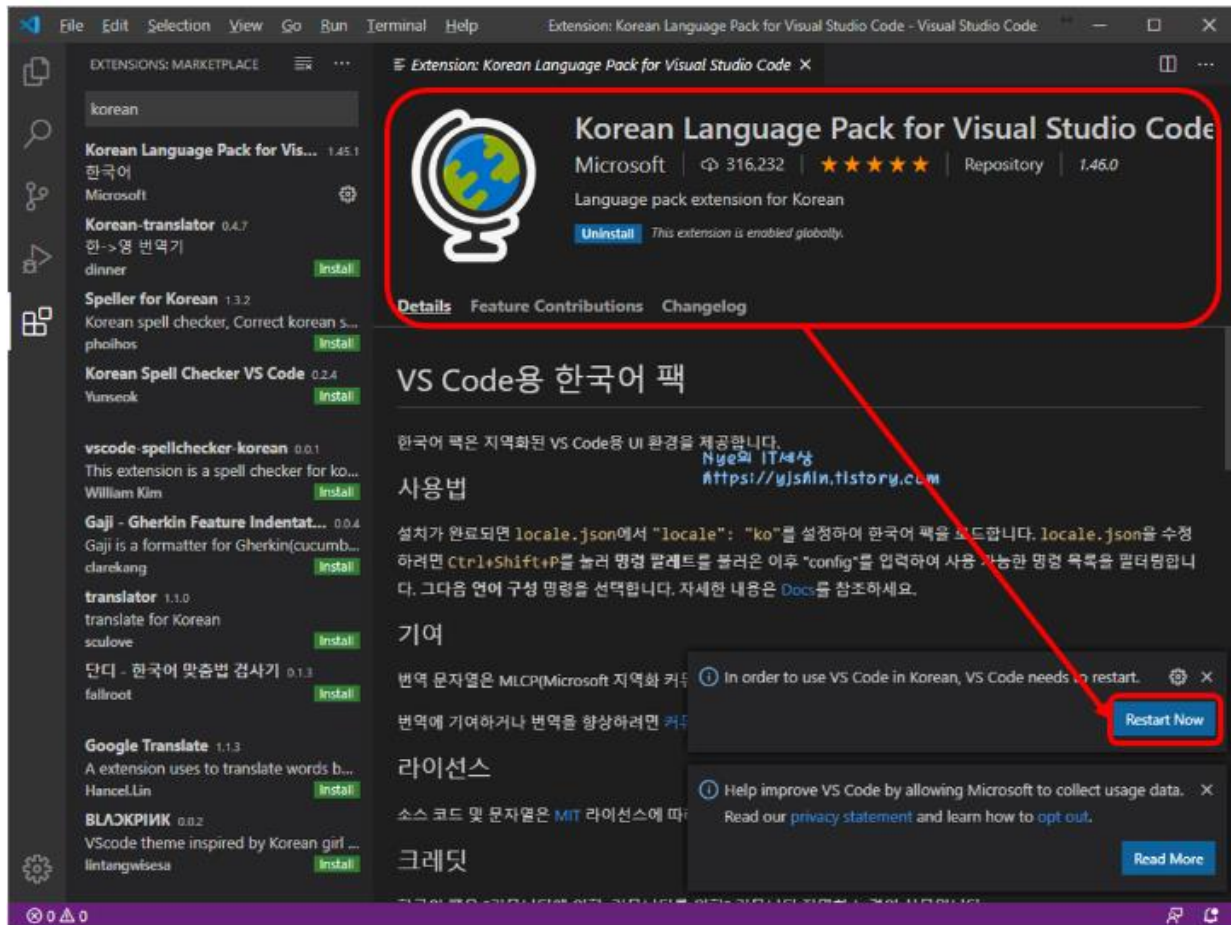
파이썬 코딩 환경

- VSCode 한글 설정



파이썬 코딩 환경

- VSCode 한글 설정



프로젝트 경로 설정

- Visual Studio Code를 수행할 때 유의할 점

✓ 위성영상처리실습 폴더를 만든다.

강의작성 참고자료	2023-07-31 오후 12:10	파일 폴더
신재생에너지자원탐사_3_2	2023-11-01 오전 11:36	파일 폴더
위성기상학_3_2	2024-08-26 오후 4:51	파일 폴더
위성영상AI프로그래밍_3_1	2024-03-27 오전 8:23	파일 폴더
위성영상처리및실습_2_2	2024-08-26 오후 3:13	파일 폴더
위성원격탐사개론_2_1	2024-06-13 오후 1:49	파일 폴더
지구환경세미나2	2023-12-05 오전 10:56	파일 폴더

프로젝트 경로 설정

- Visual Studio Code를 수행할 때 유의할 점

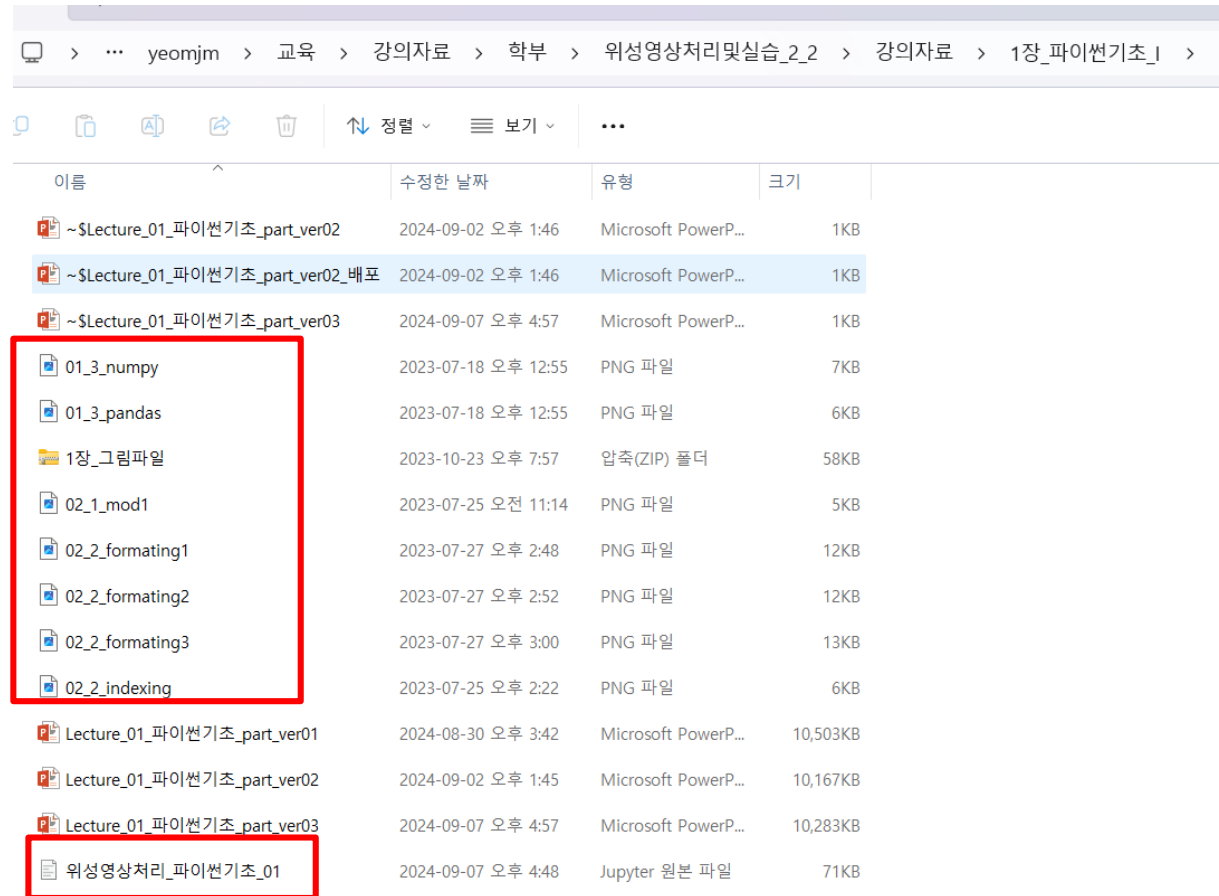
✓ 위성영상처리실습 폴더 밑에 각 강의 chapter 별로 폴더를 만든다.

1장_파이썬기초_I	2024-09-07 오후 4:49	파일 폴더
2장_파이썬기초_II	2024-09-07 오후 4:49	파일 폴더
3장_파이썬기초_III_제어문	2024-03-22 오후 4:51	파일 폴더
4장_파이썬기초_IV_입출력	2023-10-24 오전 10:13	파일 폴더
5장_파이썬기초_V_클래스	2024-03-18 오전 8:55	파일 폴더
06장_지구과학데이터_시각화_추후추가...	2023-11-23 오전 10:03	파일 폴더
6장_GK2A_입출력	2024-08-21 오전 7:55	파일 폴더
7장_구름탐지_및_지수산출	2024-04-15 오후 1:35	파일 폴더
8장_sentinel_2A_입출력	2024-08-14 오후 2:33	파일 폴더
9장_변화탐지_산불	2024-04-22 오후 1:43	파일 폴더

프로젝트 경로 설정

- Visual Studio Code를 수행할 때 유의할 점

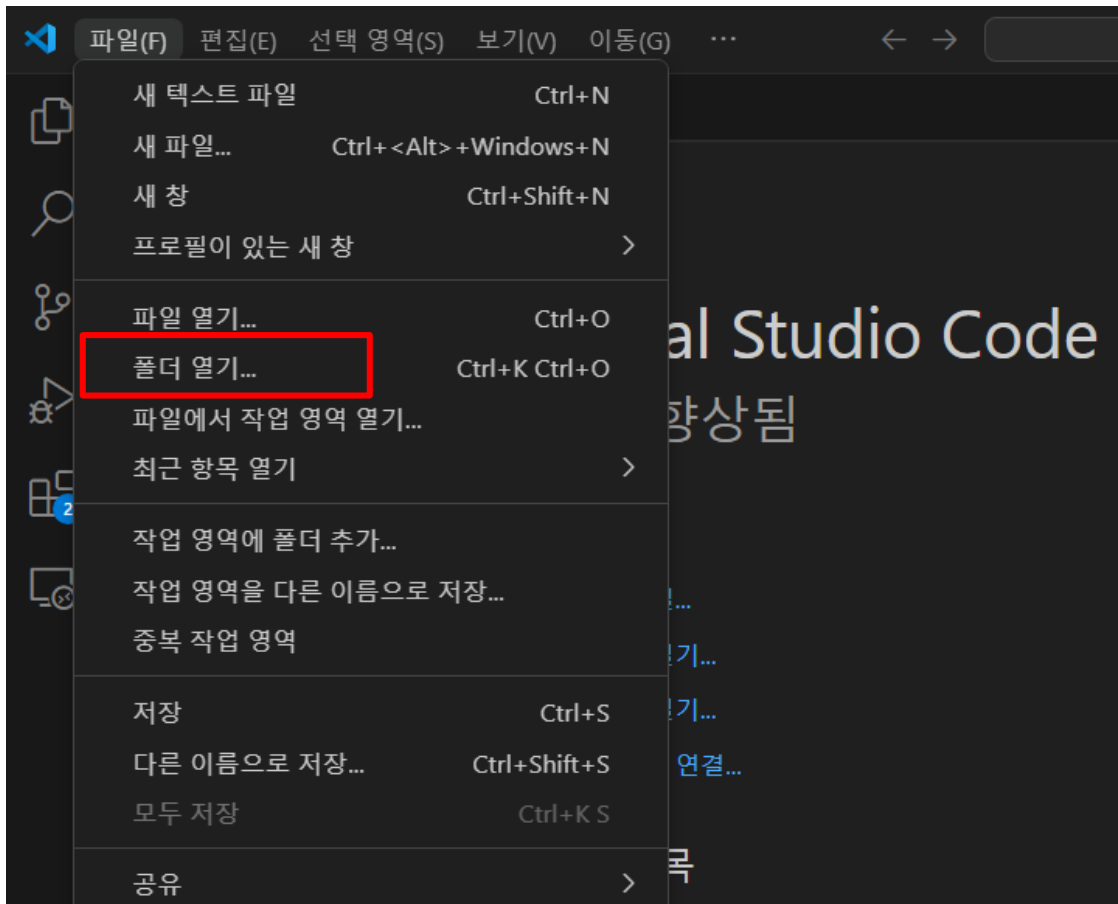
✓ 1장_파이썬기초_1 폴더에 LMS에서 받은 강의자료를 복사



이름	수정된 날짜	유형	크기
~\$Lecture_01_파이썬기초_part_ver02	2024-09-02 오후 1:46	Microsoft PowerP...	1KB
~\$Lecture_01_파이썬기초_part_ver02_배포	2024-09-02 오후 1:46	Microsoft PowerP...	1KB
~\$Lecture_01_파이썬기초_part_ver03	2024-09-07 오후 4:57	Microsoft PowerP...	1KB
01_3_numpy	2023-07-18 오후 12:55	PNG 파일	7KB
01_3_pandas	2023-07-18 오후 12:55	PNG 파일	6KB
1장_그림파일	2023-10-23 오후 7:57	압축(ZIP) 폴더	58KB
02_1_mod1	2023-07-25 오전 11:14	PNG 파일	5KB
02_2_formatting1	2023-07-27 오후 2:48	PNG 파일	12KB
02_2_formatting2	2023-07-27 오후 2:52	PNG 파일	12KB
02_2_formatting3	2023-07-27 오후 3:00	PNG 파일	13KB
02_2_indexing	2023-07-25 오후 2:22	PNG 파일	6KB
Lecture_01_파이썬기초_part_ver01	2024-08-30 오후 3:42	Microsoft PowerP...	10,503KB
Lecture_01_파이썬기초_part_ver02	2024-09-02 오후 1:45	Microsoft PowerP...	10,167KB
Lecture_01_파이썬기초_part_ver03	2024-09-07 오후 4:57	Microsoft PowerP...	10,283KB
위성영상처리_파이썬기초_01	2024-09-07 오후 4:48	Jupyter 원본 파일	71KB

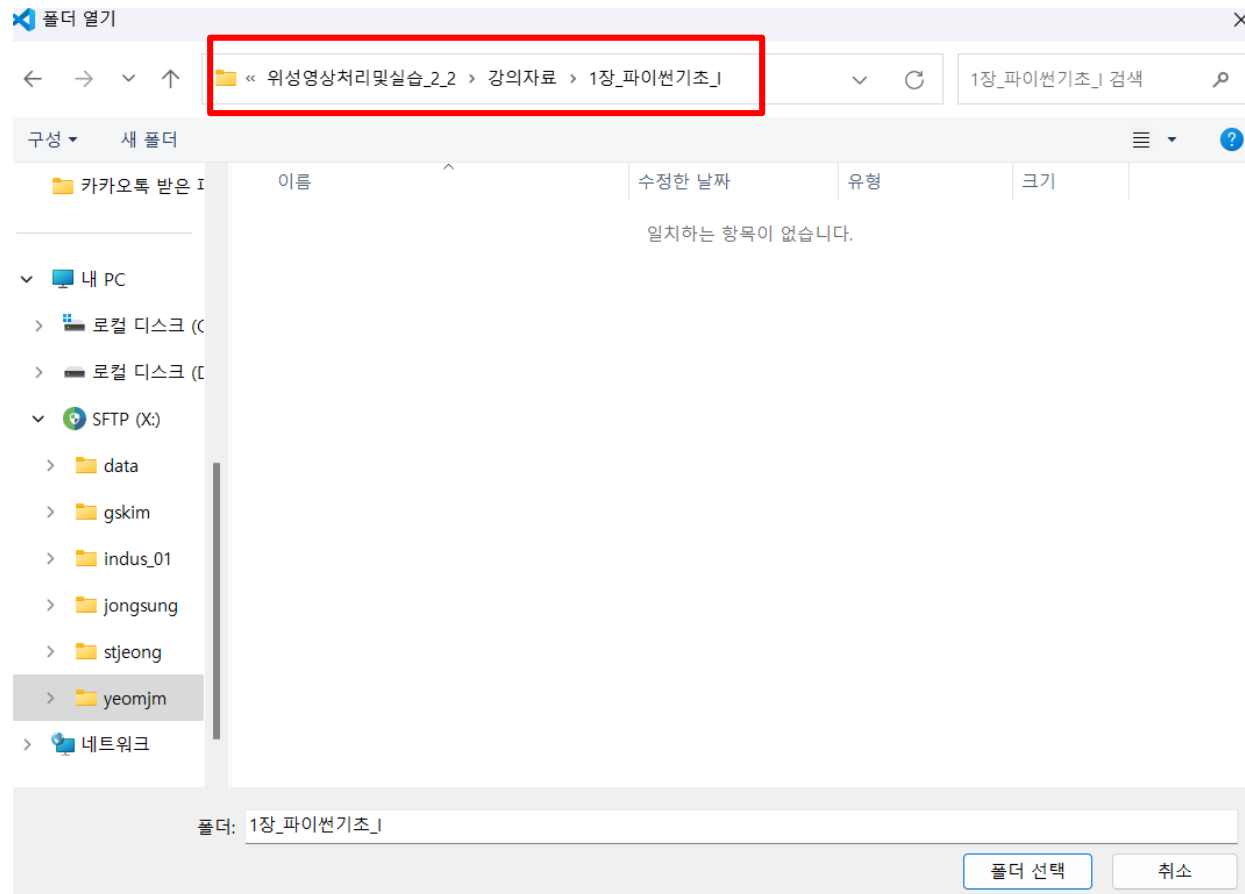
프로젝트 경로 설정

- Visual Studio Code를 수행할 때 유의할 점
 - Visual Studio Code를 실행
 - 메뉴/폴더열기 선택



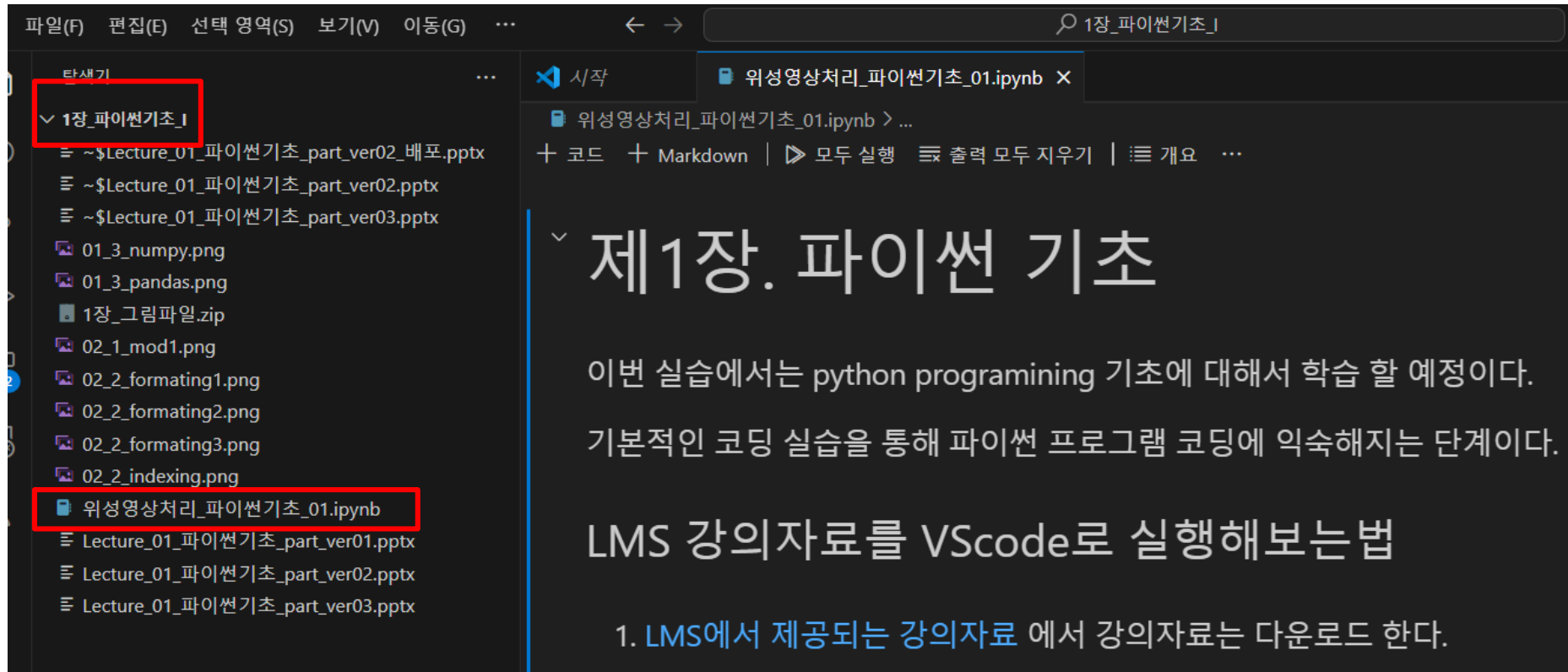
프로젝트 경로 설정

- Visual Studio Code를 수행할 때 유의할 점
 - 폴더 열기 새창이 뜬다.
 - 기존에 만들어 둔 위성영상처리및실습/1장_파이썬기초_1 폴더선택



프로젝트 경로 설정

- Visual Studio Code를 수행할 때 유의할 점
 - 프로젝트 경로에 있는 ipynb 파일을 오른쪽에 드레그 한다.

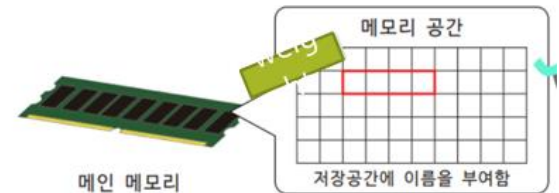


학습 참고자료

파이썬 기초

■ 데이터를 저장하는 공간

- 데이터 과학에서는 많은 데이터들을 처리하여야 함
- 어디에 데이터를 저장할 수 있을까?
- 파이썬에서는 변수variable에 데이터를 저장함
- 변수는 컴퓨터의 메모리memory 공간에 이름을 붙이는 것
- 정수, 실수, 문자열 등의 자료값을 저장할 수 있음



■ 어떻게 활용할 수 있을까

- 저장된 것을 꺼내려면 변수의 이름을 적어주면 됨
- 저장된 값은 수시로 바뀔 수 있음 → 그래서 변수(변하는 수)
- 사용된 '수'는 수치값이라는 의미가 아닌 데이터data로 이해하는 것이 더 정확함

- 문자열 인덱싱과 슬라이싱

- ✓ 인덱싱(indexing)이란 무엇인가를 '가리킨다', 슬라이싱(slicing)은 무엇인가를 '잘라 낸다'라는 의미이다. 이런 의미를 생각하면서 다음 내용을 살펴보자.

```
>>> a = "Life is too short, You need Python"
```

위 코드에서 변수 a에 저장한 문자열의 각 문자마다 번호를 매겨 보면 다음과 같다.

L	i	f	e		i	s		t	o	o		s	h	o	r	t	,		Y	o	u		n	e	e	d		P	y	t	h	o	n
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33

■ 파이썬에서의 변수 정의

- 몸무게를 weight라는 이름의 변수에 저장한다고 가정
- 파이썬 내부에 weight라는 이름의 변수가 생성되고 여기에 78.2가 저장됨

```
>>> weight = 78.2
```

- 위의 코드에 있는 '=' 기호는 '같다'는 등호의 의미가 아니라 '오른쪽의 값을 왼쪽의 weight라는 이름의 변수에 저장하라'는 의미
- 연산자 = 를 할당연산자 혹은 대입연산자라고 함

■ 변수 출력

- 변수 이름을 입력하면 변수에 저장된 값이 출력됨

```
>>> weight  
78.2
```


■ 변수 값 변경

- 체중이 78.2kg이던 사람이 76.9kg으로 변화

```
>>> weight = 78.2    # weight 변수의 최초 값
>>> weight = 76.9    # 변수가 가지고 있는 값은 변경할 수 있다
>>> weight
76.9
```

■ 여러 변수 정의

- 변수는 필요에 따라서 얼마든지 많이 만들 수 있음
- x, y는 (100, 200) 형태의 묶음으로 출력

```
>>> x = 100
>>> y = 200
>>> x, y
(100, 200)
```

▪ 동시 할당문

- a, b = 50, 20과 같이 한 줄에 여러 개의 변수를 선언하고 이 변수에 값을 동시에 할당할 수도 있음
- a+ b 연산을 하여 그 결과 값을 변수 result에 저장하고 이 값을 출력

```
In [15]: a,b=50,20
```

```
In [20]: result = a+b
```

```
In [21]: result
```

```
Out[21]: 70
```

▪ 규칙

- 문자와 숫자, 밑줄 문자(_)로 이루어져야 함(밑줄 문자 외 특수 문자 사용할 수 없음)
- 식별자의 첫 글자는 숫자로 시작할 수 없으며 중간에 공백을 가질 수 없음
- 대문자와 소문자는 구별됨 → 변수 index와 INDEX은 서로 다른 변수임
- 파이썬의 예약어(키워드)는 식별자로 사용할 수 없음

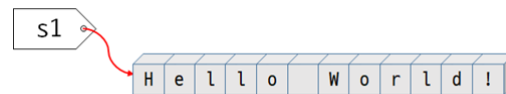
▪ 파이썬 예약어 (특정한 목적으로 사용되는 파이썬 명령어 등)

False	class	return	is	finally	None
if	for	lambda	continue	True	def
from	while	nonlocal	and	del	global
not	with	as	elif	try	or
yield	assert	else	import	pass	break
except	in	raise			

■ 문자열 저장

- 문자열 string: 텍스트 데이터도 변수에 저장할 수 있음
- 문자열은 큰따옴표("...")나 작은따옴표('...')를 이용하여 만들 수 있음
- 파이썬에서는 문자열 데이터를 '객체'라는 용어로 부르며, 변수 s1은 이 객체를 지칭하는 꼬리표로 봄

```
>>> s1 = 'Hello World!'
>>> s1
'Hello World!'
```



- 큰 따옴표, 작은 따옴표

- "..." 형태의 문자열 안에 "..." 형태의 문자열을 포함해 파이썬 인터프리터에게 넘겨주면 아래와 같이 문법 오류가 발생

```
>>> message = "철수가 "안녕"이라고 말했습니다."  
SyntaxError: invalid syntax
```

- 작은 따옴표 '...' 로 묶어주면 문자열 안 " " 이 인식됨

```
>>> message = '철수가 "안녕"이라고 말했습니다.'  
>>> message  
'철수가 "안녕"이라고 말했습니다.'  
>>> print(message)  
철수가 "안녕"이라고 말했습니다.
```

- 문자열 길이

- len() 함수

```
>>> len(s1)    # s1 문자열의 길이를 알려준다
12
```

- 문자열 연산

- + 연산 : 두 문자열을 합함

```
>>> s1 = "Hello"
>>> s2 = "World!"
>>> s1 + s2      # s1 문자열과 s2 문자열을 결합한 결과를 반환
'HelloWorld!'
>>> n1 = '100'
>>> n2 = '200'
>>> n1 + n2      # 문자열에 대한 덧셈 연산자는 문자열 이어 붙이기만 수행
'100200'
```

■ 자료형의 정의 및 유형

- 프로그램에서 사용할 수 있는 데이터의 종류를 **자료형** *data type*이라고 함
- 파이썬에서 사용할 수 있는 가장 단순한 기본 자료형은 4가지 종류가 있음

자료형	예
정수(int)	..., -3, -2, -1, 0, 1, 2, 3, ...
실수(float)	3.14, 4.28, 0.01, 123.432
문자열(str)	'Hello World', '123', "Hi"
부울형(bool)	True, False

▪ type() 함수

- 변수의 자료형을 알려면 변수의 이름에 type() 함수를 적용하면 됨
- 예를 들어서 변수 weight의 타입을 알려면 다음과 같이 하면 됨

```
>>> weight = 78.2
>>> type(weight)
<class 'float'>
```

```
>>> salary = 250
>>> type(salary)
<class 'int'>
```

```
>>> name = '홍길동'
>>> address = '부산시 광안해변로 1'
>>> type(name)
<class 'str'>
>>> type(address)
<class 'str'>
```


■ 자료형에 따른 연산

- 변수에 정수가 저장된 경우와 문자열이 저장된 경우 덧셈 연산(+)의 의미가 달라짐

```
>>> x = 10
>>> y = 10
>>> x + y
20
>>> x = 'good'
>>> y = 'morning!'
>>> x + y
'goodmorning!'
```

x, y는 각각 10, 10을 참조하는 정수형임.

x, y는 각각 'good', 'morning'을 참조하는 문자열형임.
(자료형이 변경되어 +연산이 하는 일이 달라짐)

```
>>> '23' + '56'
'2356'
>>> 23 + 56
79
```

▪ 정수와 문자열을 더할 때

- 100 뒤에 문자열 '원'을 붙여서 출력하기 위해 다음과 같이 입력

```
In [59]: 100+'원'
```

```
-----  
TypeError                                 Traceback (most recent call last)  
<ipython-input-59-4ee01bd8bc32> in <module>  
----> 1 100+'원'
```

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

정수(int)와 문자열(str)
사이에는 덧셈연산을 지
원하지 않는다는 의미.

- 100의 자료형은 정수, '원'의 자료형은 문자
- 정수+정수라면 2개의 정수를 덧셈, 문자열+문자열이라면 2개의 문자열을 연결하지만 정수+문자열은 연산할 수 없음

▪ 함수 이용

- `str()`: 정수 또는 실수 → 문자열 변환
- `int()`: 문자열 → 정수 변환
- `float()`: 문자열 → 실수 변환

```
>>> str(100)
'100'
>>> int("100")
100
```

```
>>> str(3.14)
'3.14'
>>> float("3.14")
3.14
```

▪ 사용자 입력 정보 활용

- input() 함수는 **사용자의 입력을 문자열 형태로 반환**함
- 사용자가 키보드를 이용하여 입력을 하고 엔터키를 누르면 input() 함수는 **그때까지 입력한 문자들을 모두 하나로 묶어 문자열을 만들어 반환**

```
첫 번째 정수를 입력하시오: 300  
두 번째 정수를 입력하시오: 400  
300 과 400 의 합은 700 입니다.
```

```
x = int(input("첫 번째 정수를 입력하시오: "))  
y = int(input("두 번째 정수를 입력하시오: "))  
s = x + y  
print(x, "과", y, "의 합은", s, "입니다.")
```

■ 할당 연산자

- 등호의 왼쪽이 변수가 아니면 문제가 발생함

```
>>> x = 20
>>> y = 10
>>> 100 = x + y    # 등호의 왼쪽에는 반드시 변수이름이 와야한다
File "<stdin>", line 1
SyntaxError: cannot assign to literal
```

■ 다중 할당, 동시 할당

- $x = y = 100$ 과 같이 할 경우 x, y 의 값이 모두 100으로 할당되며 이를 **다중 할당문**(multiple assignment)이라 함
- $n1, n2 = 100, 200$ 과 같이 여러 변수에 한꺼번에 여러 값을 할당할 수도 있는데 이를 **동시 할당문**(simultaneous assignment)이라 함

```
>>> x = y = 100    # 여러 변수에 동일한 값을 할당하는 다중 할당문
>>> x, y
(100, 100)
>>> n1, n2 = 100, 200  # 여러 변수에 한꺼번에 여러 값을 할당하는 동시 할당문
>>> n1, n2
(100, 200)
```

▪ 산술 연산 : 4칙 연산 등

연산자	기호	사용예	결과값
덧셈	+	7 + 4	11
뺄셈	-	7 - 4	3
곱셈	*	7 * 4	28
지수	**	7 ** 2	7 ² = 49
나눗셈(정수 나눗셈의 몫)	//	7 // 4	1
나눗셈(실수 나눗셈)	/	7 / 4	1.75
나머지	%	7 % 4	3

▪ 나눗셈 연산자

- $7/4=1.75$, $7\%4=3$: 나머지, $7//4=1$: 몫
- 짝수, 홀수 구분하기

```
number = int(input("정수를 입력하시오: "))
print(number % 2)    # 나머지로 연산의 결과가 0이면 짝수, 1이면 홀수이다
```

```
정수를 입력하시오: 28
0
```

■ 거듭제곱 연산자 : **

- 2^7 을 계산하는 파이썬 수식은 $2 ** 7 : 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2$
- 수학에서처럼 거듭제곱 연산자는 다른 연산자들보다 높은 우선순위를 가짐
- 예를 들어, $10 * 2 ** 7$ 은 20^7 이 아니라 10×2^7 로 계산되어 1280
- 예를 들어, $2 ** 2 ** 3$ 은 $(2 ** 2) ** 3$ 가 아니라 $2 ** (2 ** 3)$ 로 계산

■ 복합할당 연산자

```
>>> num = 200
>>> num += 100      # 200 + 100 연산을 수행하여 그 결과를 num에 할당
>>> num
300
>>> num -= 100      # 300 - 100 연산을 수행하여 그 결과를 num에 할당
>>> num
200
>>> num *= 20        # 200 * 20 연산을 수행하여 그 결과를 num에 할당
>>> num
4000
>>> num /= 2         # 4000 / 2 연산을 수행하여 그 결과를 num에 할당
>>> num
2000.0
```

■ 비교 연산자

- '크다' 혹은 '작다'와 같은 비교 연산은 수치 데이터를 담고 있는 두 개의 피연산자를 대상으로 크기 관계를 살펴봄
- True 혹은 False **부울**bool형으로 반환함
- **비교 연산자**comparison operator의 종류

연산자	설명	a = 100 b = 200
==	두 피연산자의 값이 같으면 True를 반환	a == b는 False
!=	두 피연산자의 값이 다르면 True를 반환	a != b는 True
>	왼쪽 피연산자가 오른쪽 피연산자보다 클 때 True를 반환	a > b는 False
<	왼쪽 피연산자가 오른쪽 피연산자보다 작을 때 True를 반환	a < b는 True
>=	왼쪽 피연산자가 오른쪽 피연산자보다 크거나 같을 때 True 반환	a >= b는 False
<=	왼쪽 피연산자가 오른쪽 피연산자보다 작거나 같을 때 True 반환	a <= b는 True

■ 비교 연산자

- `!=`, `>=` 와 같이 두 개의 문자로 이루어진 비교연산자를 사용할 때
띄어 쓰거나 순서를 바꾸면 안되며, 두 문자 사이에 공백을 넣어서도 안됨

```
>>> a, b = 100, 200 # a에는 100, b에는 200을 할당하는 동시할당문
>>> a == b
False
>>> a != b
True
>>> a > b
False
>>> a < b
True
>>> a >= b
False
>>> a > = b # 두 개의 기호로 표현된 비교 연산자는 띄어쓰면 안 된다 : ERROR
File "<stdin>", line 1
a > = b
^
SyntaxError: invalid syntax
>>> a => b # 두 개 기호로 표현된 비교 연산자의 순서를 뒤집어도 안 된다 : ERROR
File "<stdin>", line 1
a => b
^
SyntaxError: invalid syntax
```

■ 논리 연산자

- and, or, not 논리 연산자 logical operator
- 논리 and, or, not 연산을 통해 True나 False 중 하나의 값을 가지는 부울bool 값을 반환

연산자	의미
x and y	x와 y중 거짓(False)이 하나라도 있으면 거짓이 되며 모두 참(True)인 경우에만 참이다.
x or y	x나 y중에서 하나라도 참이면 참이 되며, 모두 거짓일 때만 거짓이 된다.
not x	x가 참이면 거짓, x가 거짓이면 참이 된다.

▪ random 모듈의 기능

- 모듈(module)은 변수, 함수, 클래스 등을 모아 놓은 스크립트 파일이고, 패키지(package)는 여러 모듈을 묶은 것
- random 모듈은 임의의 수를 생성하거나, 리스트나 튜플내의 요소를 무작위로 섞거나, 선택하는 함수를 포함하고 있음

```
>>> import random
>>> random.random()          # 0 이상 1 미만의 임의의 실수를 반환
0.19452357419514088
>>> random.random()          # 이 함수는 매번 다른 실수를 반환
0.6947454047320903
>>> random.randint(1, 7)      # 1 이상 7 이하(7을 포함)의 임의의 정수를 반환
3
>>> random.randrange(7)       # 0 이상 7 미만(7을 포함하지 않음)의 임의의 정수를 반환
3
>>> random.randrange(1, 7)     # 1 이상 7 미만(7을 포함안함)의 임의의 정수를 반환
6
>>> random.randrange(0, 10, 2) # 0, 2, 4, 8 중(10은 포함 안함) 하나를 반환함
2
>>> lst = [10, 20, 30, 40, 50] # 여러개의 값을 가지는 리스트를 생성함
>>> random.shuffle(lst)        # lst 리스트의 순서를 무작위로 섞음
>>> lst
[40, 50, 10, 20, 30]
>>> random.choice(lst)        # lst 리스트의 원소 중 무작위로 하나를 고름
30
```

▪ if문의 구조

- if문은 다음과 같이 구성됨

if 조건식 :	실행할 내용
----------	--------

- ✓ 조건식은 어떤 조건(condition)을 만족하는지 그렇지 않은지를 판정하는 식을 의미하며, 참(true) 또는 거짓(false)의 값을 갖는 부울(bool)형으로 평가됨
- ✓ 조건식이 참인 경우 실행이 되고 그렇지 않으면 건너뛴

■ 들여쓰기

- 들여쓰기를 이용하여서 문장들을 묶을 수 있음
→ 들여쓰기한 위치가 같으면 동일한 블록(block)에 속해 있음을 의미
- if문은 조건식이 참인 경우 콜론 뒤에 오는 블록을 모두 실행함

```
score = int(input("점수를 입력하시오: "))
if score >= 90 :           # 조건이 참일 경우 들여쓰기 블록이 모두 실행됨
    print("축하합니다.")
    print("합격입니다.")
    print("장학금도 받을 수 있습니다.")
```

```
In [9]: score = int(input('점수를 입력하시오:'))
        if score >= 90:
            print('축하합니다.')
            print('합격입니다.')
            print('장학금도 받을 수 있습니다.')

File "<ipython-input-9-eaf347a80afe>", line 4
            print('합격입니다.')
            ^
IndentationError: unexpected indent
```

주의 : 들여쓰기의 크기가 달라지면 오류!!

▪ if-elif-else 문

- if-else 문에서 조건이 거짓일 때 다른 조건을 검사할 수 있음

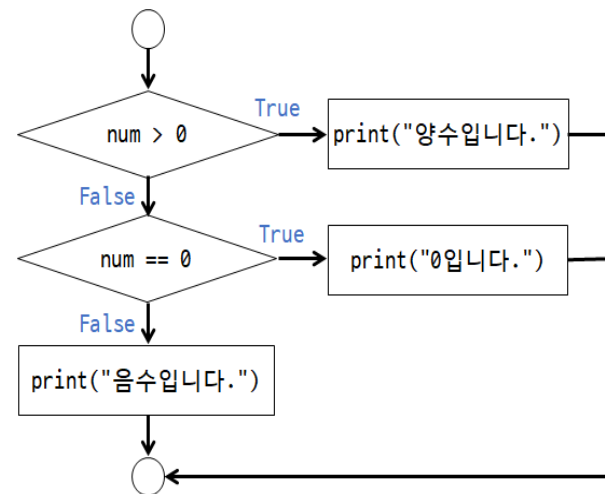
```
num = int(input("정수를 입력하시오: "))
```

```
if num > 0:
    print("양수입니다.")
elif num == 0:
    print("0입니다.")
else:
    print("음수입니다.")
```

정수를 입력하시오: 0
0입니다.

정수를 입력하시오: 10
양수입니다.

정수를 입력하시오: -10
음수입니다.



▪ 논리 연산자와 활용

- 두 조건이 동시에 만족되는지 평가하는 조건식: (조건1) **and** (조건2)
- 두 조건 중 하나 이상이 만족되는지 평가하는 조건식: (조건1) **or** (조건2)
- (실습1) 입력 받은 양의 정수가 짝수이면서 10보다 작은지 확인

```
In [9]: num = int(input('양의 정수를 입력하시오: '))
```

```
if (num % 2 == 0) and (num < 10):  
    print('짝수이면서 10보다 작습니다.')
```

```
elif (num % 2 == 0):  
    print('짝수이면서 10보다 큼니다.')
```

```
else:  
    print('홀수입니다.')
```

양의 정수를 입력하시오: 12
짝수이면서 10보다 큼니다.

▪ range() 사용

- 100번 반복 : range(100)

```
for i in range(100):  
    print("파이썬 주식회사의 방문을 환영합니다!")
```

▪ 리스트 활용

- [...]은 **리스트** list 자료형

```
for i in [1, 2, 3, 4, 5]:    # in 뒤에 리스트를 넣고 끝에 :을 넣자  
    print("i =", i)        # i 값을 출력해보자
```

- 첫 번째 반복에서 변수 i의 값은 리스트의 첫 번째 숫자인 1이 되고 print(...) 문장이 실행된다.
- 두 번째 반복에서 변수 i의 값은 리스트의 두 번째 숫자인 2가 되고 print(...) 문장이 실행된다.
- 세 번째 반복에서 변수 i의 값은 리스트의 세 번째 숫자인 3이 되고 print(...) 문장이 실행된다.
- 네 번째 반복에서 변수 i의 값은 리스트의 네 번째 숫자인 4가 되고 print(...) 문장이 실행된다.
- 다섯 번째 반복에서 변수 i의 값은 리스트의 다섯 번째 숫자인 5가 되고 print(...) 문장이 실행된다.

▪ 문자열 활용

- 문자열 적용 가능

```
for i in "Hello":    # 끝에 :이 있어야 함
    print("i =", i)  # i 값을 출력해보자
```

▪ range() 함수

- range(start, stop, step)이라고 호출하면 start에서 시작하여 (stop-1)까지 step 간격으로 정수들이 생성됨

```
for i in range(1, 6, 1):
    print(i, end = " ")    # end = " "로 지정하면 줄바꿈을 하지 않고 공백으로 나열한다.
```

```
1 2 3 4 5
```

```
for i in range(10, 0, -1):
    print(i, end = " ")
```

```
10 9 8 7 6 5 4 3 2 1
```

▪ while 문

- 어떤 조건이 만족되는 동안 반복하는 조건 제어 반복문

```
count = 1
s = 0      # s는 누적하여 더한 값을 담을 변수로 0으로 초기화 함
while count <= 10 :
    s = s + count      # 매번 count 값을 s에 더함
    count = count + 1  # 매번 count 값을 1씩 증가시킴
print("합계는", s)
```

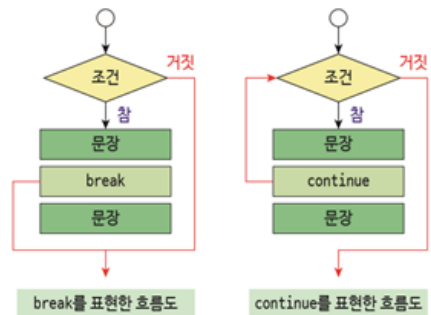
합계는 55

▪ continue 문

- continue를 이용하면, 루프를 빠져나오지 않고 continue 아래의 문장만을 건너뛴다 → 즉 반복문이 종료되는 것은 조건이 거짓일 때에만 해당함

```
st = 'I love Python Programming' # 출력을 위한 문자열
for ch in st:
    if ch in ['a','e','i','o','u', 'A','E','I','O','U']:
        continue # 모음일 경우 아래 출력을 건너뛴다
    print(ch, end='')
```

lv Pythn Prgrmmng



▪ def 예약어를 활용한 함수 정의

- 함수 안으로 값(정보)을 전달 → **인자**argument라고 함
- 함수가 보내는 값 → **반환 값**return value이라고 함

```
def calculate_area(radius):  
    area = 3.14 * radius**2  
    return area          # 이전 줄에서 구한 area 값을 호출문에 돌려준다
```

```
c_area = calculate_area(5.0)  # calculate_are() 함수가 계산한 값을 c_area에 저장
```

```
>>> print(calculate_area(5.0))  
78.5  
>>> area_sum = calculate_area(5.0) + calculate_area(10.0)  
>>> print(area_sum)
```

▪ 여러 개의 인자

- 2개의 정수 start와 end를 받아서 start에서 end까지의 합을 계산하는 함수 get_sum()을 작성

```
def get_sum(start, end):           # start, end를 매개변수로하여 인자를 받는다
    s = 0
    for i in range(start, end+1):  # start부터 end까지 정수의 합을 구함
        s += i
    return s                       # start부터 end까지 수의 합을 반환한다

print(get_sum(1, 10))             # 1에서 10까지 정수의 합 55를 출력한다
```

55

▪ 여러 개의 출력

- 큰 수부터 작은 수 순서로 정렬하는 함수 작성

```
def sort_num(n1, n2):          # 2개의 값을 받아오는 함수
    if n1 < n2:
        return n1, n2         # n1이 더 작으면 n1, n2 순서로 반환
    else:
        return n2, n1         # n2가 더 작으면 n2, n1 순서로 반환

print(sort_num(110, 210))      # 110과 210을 함수의 인자로 전달하고 반환되는 값을 출력
print(sort_num(2100, 80))
```

```
(110, 210)
(80, 2100)
```

■ 지역변수

- 함수 안에서 생성되는 변수를 **지역변수** local variable라고 함
- 이 지역변수 `counter`가 참조하는 값 200은 전역변수가 참조하는 값 100과 별개의 메모리에 있으며, 지역변수는 함수가 종료되면 사라짐

```
def print_counter():  
    counter = 200  
    print('counter =', counter) # 함수 내부의 counter 값  
  
counter = 100  
print_counter()  
print('counter =', counter)    # 함수 외부의 counter 값
```

```
counter = 200  
counter = 100
```

▪ 함수 내에서 전역변수 활용

- 만일 함수 내부에서 새로 변수를 만들지 않고 전역변수인 counter를 불러서 사용 → global이라는 키워드를 사용

```
def print_counter():  
    global counter      # 함수 외부의 전역변수 counter를 사용하겠다는 선언  
    counter = 200  
    print('counter =', counter) # 함수 내부의 counter 값  
  
counter = 100  
print_counter()  
print('counter =', counter)    # 함수 외부의 counter 값  
  
counter = 200  
counter = 200
```

전역변수를 많이 사용할 경우 프로그램에서 어느 부분에서 오류가 발생했는지 추적하기가 어려움

▪ 디폴트 인자

- 인자에 기본값을 넣어줌 → 디폴트 인자

```
def order(num, pickle = True, onion = True) :  
    print('햄버거 {0} 개 - 피클 {1}, 양파 {2}'.format(num, pickle, onion))
```

```
order(1, pickle = False, onion = True)
```

```
order(2)      # 햄버거 2개를 주문, 디폴트로 pickle, onion 값은 True임
```

```
햄버거 1 개 - 피클 False, 양파 True
```

```
햄버거 2 개 - 피클 True, 양파 True
```

▪ 키워드 인자

- 파이썬에서 대부분의 인자들은 함수 호출 시에 위치에 의하여 구별됨

```
def power(base, exponent):  
    return base**exponent # base가 밑이고, exponent가 지수값이다.
```

```
>>> power(2, 10) # 2의 10승을 반환한다  
1024  
>>> power(10, 2) # 10의 2승을 반환한다  
100
```

어떤 값이 밑이고 지수인지
헛갈릴 수 있다 -> 오류의
원인이 된다

```
>>> power(base=2, exponent=10) # 2의 10승을 반환한다  
1024  
>>> power(exponent=10, base=2) # 2의 10승을 반환한다  
1024
```

밑(base)과
지수(exponent)를
명시를 해주므로
헛갈리지 않는다.

▪ 재귀 함수

- 재귀함수^{recursion}란 함수 내부에서 자기 자신을 호출하는 함수를 말함
- 팩토리얼 식과 정의 : $n! = n * (n - 1)!$

```
1! = 1
2! = 2 * 1
3! = 3 * 2 * 1
...
n! = n * (n-1) * (n-2) * ... * 2 * 1
```

```
def factorial(n):           # n!의 재귀적 구현
    if n <= 1 :             # 종료 조건이 반드시 필요하다
        return 1
    else :
        return n * factorial(n-1)  # n * (n-1)! 정의에 따른 구현

print('4! = ', factorial(4))  # 인자로 4를 넣어 호출
```

강의 참조자료

- 점프 투 파이썬 위키독스
 - ✓ <https://wikidocs.net/book/1>
- 파이썬 기초 관련 인터넷 및 Blog 자료